LogForge Open-Source Version - Technical Requirements Specification

Version: 1.0
Date: 2025-11-11
License: Apache License 2.0
Target: AI Coding Agent Development

Executive Summary

LogForge is a synthetic event log generator designed to produce realistic log data from various systems. This document specifies requirements for the **open-source version**, which provides a complete, production-ready log generation system with API-first architecture, template-based event generation, and entity registry management.

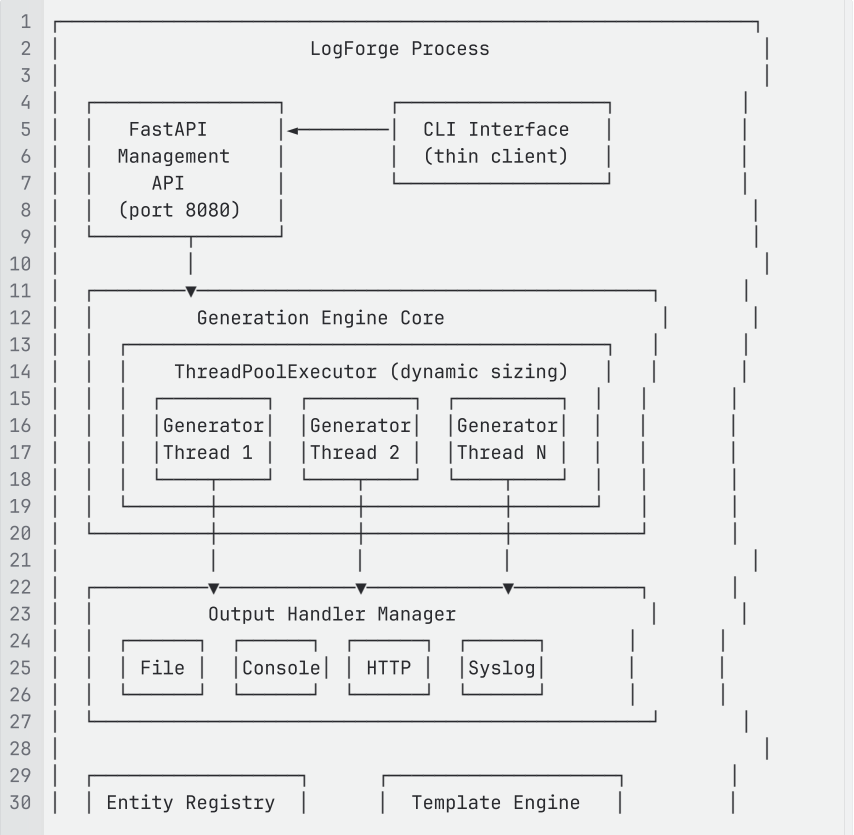
Key Principles:

- API-first architecture (FastAPI embedded server)
- Template-based with zero-code configuration
- File-based persistence for simplicity
- Thread-based concurrent generation
- Smart error handling with recovery
- Comprehensive observability

---

1. System Architecture

1.1 Core Architecture



1.2 Component Responsibilities

FastAPI Management API:

- Embedded server running in background thread
- Provides REST endpoints for all operations
- Health checks, metrics, and monitoring
- Optional API key authentication

CLI Interface:

- Thin wrapper around API calls
- Handles user interaction and output formatting
- Connects to local or remote API

Generation Engine Core:

- Manages generator lifecycle (STOPPED, STARTING, RUNNING, DEGRADED, ERROR)
- ThreadPoolExecutor with dynamic thread pool sizing
- Coordinates template rendering and output routing
- Implements smart error recovery

Entity Registry:

- File-based storage (YAML) with in-memory cache
- Provides realistic entities (users, devices, services) to templates
- Auto-save with configurable flush interval

Template Engine:

- Jinja2-based rendering with custom filters
- Faker library integration for synthetic data
- Template validation and metadata parsing

Output Handlers:

- Pluggable output destinations (file, console, HTTP, TCP, syslog)
- Retry logic with exponential backoff
- Event buffering during outages

---

2. API Specification

2.1 API Server Configuration

Lifecycle: Single process with embedded FastAPI server running in background thread

Configuration:

```yaml
api:
  enabled: true              # Can disable for headless mode
  host: 127.0.0.1           # Listen address
  port: 8080                # Listen port
```

```yaml
  auth:
    enabled: false          # Optional API key authentication
    key: null               # API key (generate if enabled)
```

Authentication:

- Default: No authentication (localhost trust)
- Optional: API key in `Authorization: Bearer <key>` header
- Generated on first run if `auth.enabled: true`

2.2 API Endpoints

Health & Status

GET /api/health

```
Response 200:
{
  "status": "healthy|degraded|unhealthy",
  "uptime": 3600,
  "generators": {
    "total": 5,
    "running": 4,
    "degraded": 1,
    "error": 0
  },
  "entity_registry": "healthy",
  "template_cache": "healthy"
}
```

GET /api/status

```
Response 200:
{
  "uptime": 3600,
  "version": "1.0.0",
  "generators": [
    {
      "name": "windows_security",
      "state": "RUNNING",
      "template": "microsoft/windows/eventlog/security",
      "events_generated": 12450,
      "errors": 0,
      "uptime": 3540
    }
  ],
  "system": {
    "cpu_percent": 15.2,
    "memory_mb": 245,
    "threads": 8
  }
}
```

GET /api/metrics

- Returns Prometheus-compatible metrics
- Counters: events_generated_total, errors_total
- Gauges: generators_running, memory_usage_bytes
- Histograms: template_render_seconds, output_latency_seconds

Generator Management

GET /api/generators

```
Response 200:
{
```

```
  "generators": [
    {
      "name": "windows_security",
      "enabled": true,
      "state": "RUNNING",
      "template": "microsoft/windows/eventlog/security"
    }
  ]
}
```

GET /api/generators/{name}

```
Response 200:
{
  "name": "windows_security",
  "state": "RUNNING",
  "template": "microsoft/windows/eventlog/security",
  "enabled": true,
  "frequency": {
    "base_rate": 10,
    "current_rate": 20
  },
  "outputs": ["default_file", "console_json"],
  "statistics": {
    "events_generated": 12450,
    "errors": 0,
    "uptime": 3540,
    "last_event": "2025-11-11T10:15:23Z"
  }
}
```

POST /api/generators/{name}/start

```
Request: {}
Response 200:
{
  "name": "windows_security",
  "state": "STARTING",
  "message": "Generator starting"
}
```

POST /api/generators/{name}/stop

```
Response 200:
{
  "name": "windows_security",
  "state": "STOPPING",
  "message": "Generator stopping"
}
```

POST /api/generators/{name}/restart

```
Response 200:
{
  "name": "windows_security",
  "state": "STARTING",
  "message": "Generator restarting"
}
```

Template Management

GET /api/templates

```
Response 200:
{
  "templates": [
    {
      "id": "microsoft/windows/eventlog/security",
      "name": "Windows Security Event Log",
      "vendor": "Microsoft",
```

```
      "product": "Windows",
      "data_source": "Event Log",
      "version": "1.0.0",
      "local": true,
      "remote_version": "1.0.1"
    }
  ]
}
```

GET /api/templates/{template_id}

```
Response 200:
{
  "id": "microsoft/windows/eventlog/security",
  "name": "Windows Security Event Log",
  "description": "Generates Windows Security Event Log entries",
  "vendor": "Microsoft",
  "product": "Windows",
  "data_source": "Event Log",
  "version": "1.0.0",
  "format": "xml",
  "local": true,
  "remote_version": "1.0.1",
  "metadata": {
    "author": "LogForge Community",
    "updated": "2025-10-15"
  }
}
```

Entity Management

GET /api/entities

```
Response 200:
{
  "organization": {
    "name": "Acme Corp",
    "domain": "acme.com"
  },
  "users": 150,
  "devices": 75,
  "services": 12
}
```

GET /api/entities/{type}

Where type: users|devices|services

```
Response 200:
{
  "type": "users",
  "count": 150,
  "entities": [
    {
      "username": "jsmith",
      "email": "jsmith@acme.com",
      "department": "Engineering"
    }
  ]
}
```

---

3. Configuration Management

3.1 Configuration File Format

Location: `~/.logforge/config.yaml` or `./config.yaml`

Template Configuration Section:

```yaml
# Template Settings
templates:
  local_path: ~/.logforge/templates
  default_path: ~/.logforge/templates/default  # Community templates
  custom_path: ~/.logforge/templates/custom    # User templates
  precedence: custom_first                      # custom_first, default_first, explicit
  community_api_url: https://api.logforge.io/v1
  auto_update_check: true
  cache_ttl: 3600                               # seconds

  # Customization workflow settings
  auto_backup_on_customize: true                # Backup default before customizing
  diff_tool: auto                               # auto, vimdiff, meld, custom command
```

Precedence Options:

- `custom_first` (default): Check custom/ first, fall back to default/
- `default_first`: Check default/ first, fall back to custom/ (unusual)
- `explicit`: Require namespace prefix (default: or custom:) in generator config

Complete Schema:

```yaml
# LogForge Configuration
version: "1.0"

# Core Engine Settings
engine:
  max_generators: 10              # null for unlimited based on CPU
  thread_pool_size: null          # null for auto (CPU cores × 5)
  log_level: INFO                 # DEBUG, INFO, WARNING, ERROR, CRITICAL

# API Server Settings
api:
  enabled: true                   # Disable for headless/embedded mode
  host: 127.0.0.1                # Listen address
  port: 8080                      # Listen port
  auth:
    enabled: false                # Enable API key authentication
    key: null                     # Auto-generated if enabled

# Entity Registry Settings
entity_registry:
  path: ~/.logforge/entities.yaml
  auto_save: true
  save_interval: 60               # seconds
  backup_enabled: true
  backup_count: 3                 # Keep N backups

# Template Settings
templates:
  local_path: ~/.logforge/templates
  community_api_url: https://api.logforge.io/v1
  auto_update_check: true
  cache_ttl: 3600                 # seconds

# Application Logging
logging:
  level: INFO
  file: ~/.logforge/logforge.log
  rotation:
    max_size: 50MB
    backup_count: 5
  format: "%(asctime)s - %(name)s - %(levelname)s - %(message)s"
```

```yaml

# Output Handler Settings
outputs:
  retry:
    max_attempts: -1             # -1 = unlimited, 0 = no retry
    retry_interval: 5            # seconds
    backoff_multiplier: 2.0      # exponential backoff
    max_backoff: 300             # max seconds between retries
  buffer_size: 10000             # events to buffer during outage

  # Output Destination Definitions
  definitions:
    - name: default_file
      type: file
      path: /var/log/logforge/{generator}.log
      rotation:
        type: size                # size or time
        max_size: 100MB
        max_age: 7d              # for time-based rotation
        compress: true

    - name: console_json
      type: console
      format: json                # json or text

    - name: syslog_server
      type: syslog
      host: syslog.example.com
      port: 514
      protocol: tcp               # tcp or udp
      facility: local0

    - name: http_collector
      type: http
      url: https://collector.example.com/events
      method: POST
      headers:
        Authorization: Bearer ${API_TOKEN}
      batch_size: 100
      batch_interval: 5           # seconds

# Generator Definitions
generators:
  - name: windows_security
    template: microsoft/windows/eventlog/security
    enabled: true
    frequency:
      base_rate: 10               # events per second
      variation:
        - days: [1,2,3,4,5]       # Monday-Friday
          time: "09:00-17:00"
          multiplier: 2.0
        - days: [6,7]             # Weekend
          multiplier: 0.5
    outputs: [default_file, console_json]

  - name: palo_alto_traffic
    template: paloalto/firewall/traffic
    enabled: true
    frequency:
      base_rate: 50
    outputs: [default_file, syslog_server]
```

3.2 Configuration Initialization

Command: `logforge init`

Behavior:

- Creates `~/.logforge/` directory structure

- Generates default `config.yaml` with sensible defaults
- Creates empty `entities.yaml` with organization structure
- Creates `templates/` directory
- Optionally runs interactive wizard with `--interactive` flag

Interactive Wizard Prompts:

1. Organization name and domain
2. Log output directory preference
3. Default event generation rate
4. API server port (default 8080)
5. Template installation (install starter pack?)

---

4. Template System

4.1 Template Structure

Directory Layout:

```
~/.logforge/templates/
├── default/                    # Community templates (managed by LogForge)
│   ├── microsoft/
│   │   └── windows/
│   │       └── eventlog/
│   │           └── security/
│   │               ├── template.j2
│   │               ├── metadata.yaml
│   │               └── examples/
│   │                   └── sample_output.xml
│   └── paloalto/
│       └── firewall/
│           └── traffic/
│               ├── template.j2
│               └── metadata.yaml
│
└── custom/                     # User templates (never touched by updates)
    ├── microsoft/              # Customized versions of default templates
    │   └── windows/
    │       └── eventlog/
    │           └── security/   # User's customized version
    │               ├── template.j2
    │               └── metadata.yaml
    ├── paloalto/               # User modifications
    │   └── firewall/
    │       └── traffic/
    │           └── template.j2
    └── acme/                   # Completely custom vendor
        └── custom_app/
            ├── template.j2
            └── metadata.yaml
```

Template Resolution (Precedence System):

When a generator references `microsoft/windows/eventlog/security`, the system resolves in this order:

1. **Check** `custom/microsoft/windows/eventlog/security` (user customization)

2. **Fall back to** `default/microsoft/windows/eventlog/security` (community template)
3. **Error** if neither exists

This allows users to override community templates with their own versions while preserving the ability to update defaults.

Metadata File (`metadata.yaml`):

```yaml
id: microsoft/windows/eventlog/security
name: Windows Security Event Log
description: Generates realistic Windows Security Event Log entries
vendor: Microsoft
product: Windows
data_source: Event Log
version: 1.0.0              # Only for default/ templates
format: xml
author: LogForge Community
created: 2025-10-01
updated: 2025-10-15
base_template: null         # For custom/ templates, references default version
tags:
  - windows
  - security
  - authentication
variables:
  - name: event_id
    type: integer
    description: Windows Event ID
  - name: username
    type: string
    description: User account name
```

4.2 Template Rendering Context

Templates have access to:

Registry Functions:

```
{{ registry.get_random_user() }}          {# Returns random user dict #}
{{ registry.get_random_device() }}        {# Returns random device dict #}
{{ registry.get_random_service() }}       {# Returns random service dict #}
{{ registry.get_user('jsmith') }}         {# Get specific user #}
{{ registry.get_device('ws001') }}        {# Get specific device #}
{{ registry.get_service('web_app') }}     {# Get specific service #}
{{ registry.get_organization() }}         {# Organization dict #}
{{ registry.get_organization_field('domain') }}  {# Specific field #}
{{ registry.get_organization_contact('admin') }} {# Contact info #}
```

Faker Library (via `fake` object):

```
{{ fake.name() }}                         {# Random person name #}
{{ fake.ipv4() }}                         {# Random IPv4 address #}
{{ fake.ipv6() }}                         {# Random IPv6 address #}
{{ fake.mac_address() }}                  {# Random MAC address #}
{{ fake.url() }}                          {# Random URL #}
{{ fake.user_agent() }}                   {# Random user agent string #}
{{ fake.file_path() }}                    {# Random file path #}
{{ fake.uuid4() }}                        {# Random UUID #}
```

Built-in Filters:

```
{{ now() }}                               {# Current timestamp #}
{{ now() | format_datetime('%Y-%m-%d') }} {# Formatted timestamp #}
```

```
{{ random_int(1, 100) }}                {# Random integer #}
{{ random_choice(['A', 'B', 'C']) }}    {# Random selection #}
```

4.3 Example Template

File: `default/microsoft/windows/eventlog/security/template.j2`

```
<Event xmlns="http://schemas.microsoft.com/win/2004/08/events/event">
  <System>
    <Provider Name="Microsoft-Windows-Security-Auditing" />
    <EventID>{{ random_choice([4624, 4625, 4634, 4648]) }}</EventID>
    <TimeCreated SystemTime="{{ now() | format_datetime('%Y-%m-%dT%H:%M:%S.%fZ') }}" />
    <Computer>{{ registry.get_random_device().hostname }}.{{ registry.get_organization_field('domain') }}</Computer>
  </System>
  <EventData>
    <Data Name="TargetUserName">{{ registry.get_random_user().username }}</Data>
    <Data Name="TargetDomainName">{{ registry.get_organization_field('domain').upper() }}</Data>
    <Data Name="IpAddress">{{ fake.ipv4() }}</Data>
    <Data Name="WorkstationName">{{ registry.get_random_device().hostname }}</Data>
  </EventData>
</Event>
```

4.4 Template Validation

Validation Checks:

1. Valid Jinja2 syntax
2. Metadata file present and valid
3. Referenced registry functions exist
4. No unsafe operations (eval, exec, file access)
5. Output format matches declared format
6. For custom templates: base_template reference valid (if specified)

Command: `logforge templates validate <path>`

4.5 Template Customization Workflow

Making a Custom Version:

```
# Copy default template to custom for editing
logforge templates customize microsoft/windows/eventlog/security

# Output:
# Copying default/microsoft/windows/eventlog/security → custom/
# Template now editable at:
~/.logforge/templates/custom/microsoft/windows/eventlog/security
#
# Your generators using 'microsoft/windows/eventlog/security' will now automatically
# use this custom version. The default version remains available for updates.
```

Updating Custom Templates When Defaults Change:

```
# See what changed in default version
logforge templates diff microsoft/windows/eventlog/security

# Output:
# Comparing custom vs default v1.0.2
#
```

```
# --- custom/microsoft/windows/eventlog/security/template.j2
# +++ default/microsoft/windows/eventlog/security/template.j2
# @@ -5,7 +5,7 @@
#    <EventID>{{ random_choice([4624, 4625, 4634, 4648]) }}</EventID>
# -  <TimeCreated SystemTime="{{ now() }}" />
# +  <TimeCreated SystemTime="{{ now() | format_datetime('%Y-%m-%dT%H:%M:%S.%fZ') }}" />

# Merge default changes into custom (interactive)
logforge templates merge microsoft/windows/eventlog/security

# Or manually update custom version
vim ~/.logforge/templates/custom/microsoft/windows/eventlog/security/template.j2
```

Reverting to Default:

```
# Remove custom version to use default again
logforge templates revert microsoft/windows/eventlog/security

# Output:
# Removed custom/microsoft/windows/eventlog/security
# Generators will now use default version (v1.0.2)
```

---

5. Entity Registry

5.1 Entity Storage Format

File: `~/.logforge/entities.yaml`

```yaml
organization:
  name: Acme Corporation
  domain: acme.com
  contacts:
    admin: admin@acme.com
    security: security@acme.com
  attributes:
    industry: Technology
    employee_count: 500

users:
  - username: jsmith
    email: jsmith@acme.com
    full_name: John Smith
    department: Engineering
    role: Senior Developer
    attributes:
      employee_id: EMP001
      manager: mjones

  - username: mjones
    email: mjones@acme.com
    full_name: Mary Jones
    department: Engineering
    role: Engineering Manager
    attributes:
      employee_id: EMP002

devices:
  - hostname: ws001
    ip_address: 192.168.1.10
    mac_address: 00:11:22:33:44:55
    os: Windows 10
    owner: jsmith
    attributes:
      asset_tag: AT-001
      location: Building A
```

```yaml

  - hostname: web-srv-01
    ip_address: 10.0.1.50
    mac_address: AA:BB:CC:DD:EE:FF
    os: Ubuntu 22.04
    type: server
    attributes:
      datacenter: DC1
      rack: R12

services:
  - name: web_app
    description: Main Web Application
    url: https://app.acme.com
    port: 443
    protocol: https

  - name: api_gateway
    description: API Gateway Service
    url: https://api.acme.com
    port: 443
    protocol: https
```

5.2 Entity Management Operations

CLI Commands:

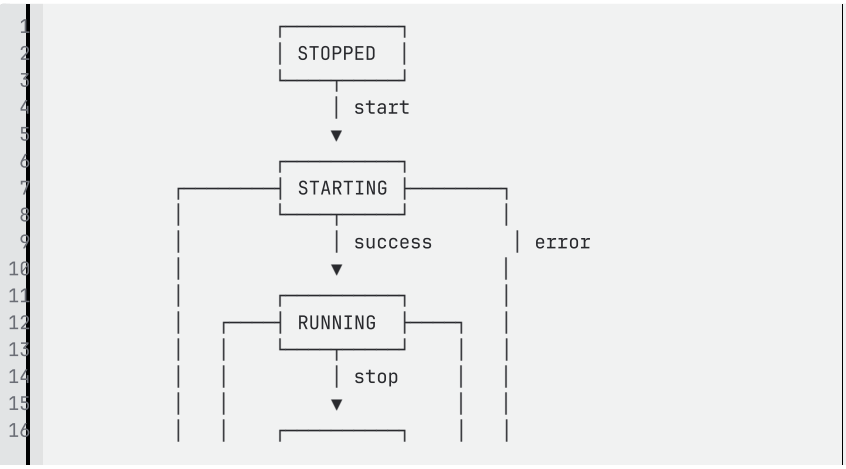
```bash
# List entities
logforge entities list                    # All entities summary
logforge entities list --type users       # Specific type

# Show specific entity
logforge entities show user jsmith
logforge entities show device ws001

# Add entity (interactive)
logforge entities add user
logforge entities add device

# Import/Export
logforge entities import entities.yaml
logforge entities export backup.yaml

# Validate
logforge entities validate
```

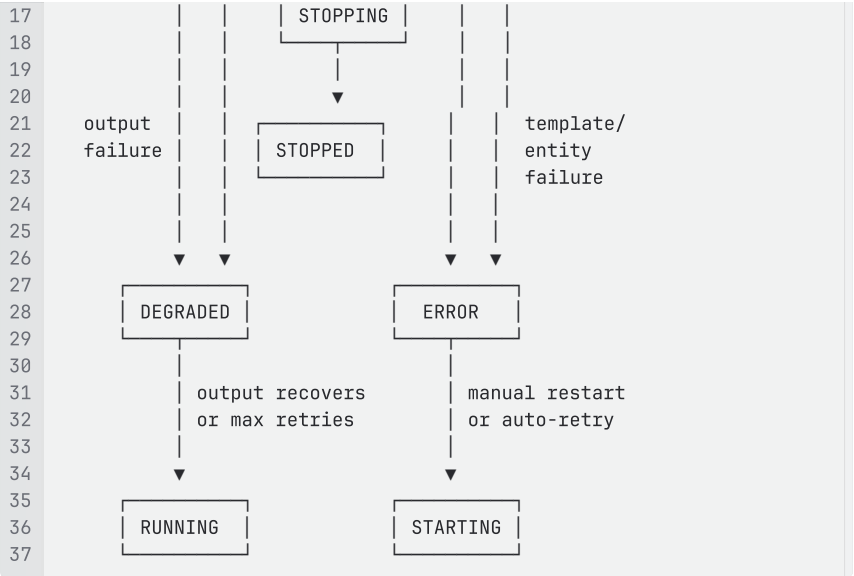
API Endpoints: See section 2.2

---

6. Generator Lifecycle & Error Handling

6.1 Generator State Machine





6.2 Error Handling Behaviors

Template Rendering Failure

Cause: Invalid template syntax, missing entity, Jinja2 error

Behavior:

1. Generator transitions to ERROR state
2. Log detailed error with template location, line number, and context
3. Prevent new generators from using this template
4. Continue running other generators
5. **Recovery**: Smart retry on transient errors (missing entity), stay in ERROR on config errors (syntax)

Smart Retry Logic:

```
if error_type == "EntityNotFound":
    # Transient - entity might be added later
    retry_after = 60  # seconds
    max_retries = 5
elif error_type == "TemplateSyntaxError":
    # Configuration error - don't retry
    stay_in_error_state = True
```

Output Destination Unavailable

Cause: Network down, file system full, permission denied

Behavior:

1. Generator transitions to DEGRADED state
2. Begin retry logic with exponential backoff
3. Buffer events in memory (up to configured `buffer_size`)
4. When buffer full: drop oldest events, log warning
5. **Recovery**: When output recovers, flush buffer and transition to RUNNING

Retry Configuration:

```
outputs:
  retry:
    max_attempts: -1          # -1 = unlimited
    retry_interval: 5         # initial seconds
```

```
    backoff_multiplier: 2.0    # exponential backoff
    max_backoff: 300           # max 5 minutes between retries
  buffer_size: 10000           # events
```

Retry Sequence:

```
Attempt 1: Wait 5 seconds
Attempt 2: Wait 10 seconds  (5 × 2.0)
Attempt 3: Wait 20 seconds  (10 × 2.0)
Attempt 4: Wait 40 seconds  (20 × 2.0)
Attempt 5: Wait 80 seconds  (40 × 2.0)
Attempt 6: Wait 160 seconds (80 × 2.0)
Attempt 7: Wait 300 seconds (capped at max_backoff)
Attempt 8+: Wait 300 seconds (continue until recovery or manual stop)
```

Entity Registry Corruption

Cause: Invalid YAML, schema violation, file permission error

Behavior:

1. All generators using entities transition to ERROR state
2. Prevent new generators from starting
3. Log validation errors with details
4. Attempt to load backup if `backup_enabled: true`
5. **Recovery**: User fixes entities.yaml, runs `logforge entities validate`, restarts generators

Validation Checks:

- Valid YAML syntax
- Required fields present (organization, users, devices, services)
- Valid data types (strings, integers, lists, dicts)
- No duplicate usernames/hostnames
- Valid email formats, IP addresses, MAC addresses

6.3 Logging

Application Log Location: `~/.logforge/logforge.log`

Log Levels:

- **DEBUG**: Template rendering details, entity lookups, internal operations
- **INFO**: Generator state changes, configuration loaded, API requests
- **WARNING**: Retry attempts, buffer near full, deprecated config
- **ERROR**: Template rendering failures, output failures, entity errors
- **CRITICAL**: System failures, unable to start API, corrupted config

Example Log Entries:

```
2025-11-11 10:15:23 - logforge.engine - INFO - Generator 'windows_security' starting
2025-11-11 10:15:24 - logforge.engine - INFO - Generator 'windows_security' transitioned to RUNNING
2025-11-11 10:16:30 - logforge.outputs.file - WARNING - File output failed: [Errno 28] No space left on device
2025-11-11 10:16:30 - logforge.engine - WARNING - Generator 'windows_security' transitioned to DEGRADED
2025-11-11 10:16:35 - logforge.outputs.file - INFO - Retry attempt 1/∞ in 5 seconds
```

```
2025-11-11 10:17:00 - logforge.outputs.file - INFO - Output recovered, flushing 150 buffered events
2025-11-11 10:17:01 - logforge.engine - INFO - Generator 'windows_security' transitioned to RUNNING
```

---

7. Output Handlers

7.1 File Output Handler

Configuration:

```yaml
- name: default_file
  type: file
  path: /var/log/logforge/{generator}.log
  rotation:
    type: size              # size or time
    max_size: 100MB         # for size-based
    max_age: 7d             # for time-based (7d, 24h, etc)
    compress: true          # gzip rotated files
```

Features:

- Variable substitution in path: `{generator}`, `{date}`, `{timestamp}`
- Atomic file operations
- Separate file per generator by default
- Rotation triggers: size threshold OR time interval
- Compressed rotated files (.gz)

Rotation Behavior:

```
# Size-based rotation at 100MB
app.log          (current, 95MB)
app.log.1.gz     (100MB compressed)
app.log.2.gz     (100MB compressed)
app.log.3.gz     (100MB compressed)

# Time-based rotation daily
app.log                    (current, today)
app-2025-11-10.log.gz     (yesterday)
app-2025-11-09.log.gz     (2 days ago)
```

7.2 Console Output Handler

Configuration:

```yaml
- name: console_json
  type: console
  format: json            # json or text
  stream: stdout          # stdout or stderr
```

Output Formats:

- **json**: One JSON object per line (JSONL)
- **text**: Human-readable formatted output

7.3 HTTP Output Handler

Configuration:

```yaml
- name: http_collector
  type: http
  url: https://collector.example.com/events
  method: POST
```

```
  headers:
    Authorization: Bearer ${API_TOKEN}
    Content-Type: application/json
  batch_size: 100         # events per request
  batch_interval: 5       # seconds
  timeout: 30            # request timeout
```

Features:

- Event batching for efficiency
- Time-based or size-based batch triggers
- Environment variable substitution in headers: `${VAR_NAME}`
- Automatic JSON array wrapping for batches

7.4 TCP Output Handler

Configuration:

```
- name: tcp_server
  type: tcp
  host: 192.168.1.100
  port: 9000
  delimiter: "\n"         # Event delimiter
  keepalive: true        # TCP keepalive
```

7.5 Syslog Output Handler

Configuration:

```
- name: syslog_server
  type: syslog
  host: syslog.example.com
  port: 514
  protocol: tcp           # tcp or udp
  facility: local0        # syslog facility
  severity: info         # syslog severity
  format: rfc5424        # rfc5424 or rfc3164
```

Syslog Format:

- **RFC 5424** (modern): `<PRI>VERSION TIMESTAMP HOSTNAME APP-NAME PROCID MSGID STRUCTURED-DATA MSG`
- **RFC 3164** (legacy): `<PRI>TIMESTAMP HOSTNAME TAG: MSG`

---

8. Community Integration

8.1 Community API Client

Purpose: Discover and download templates from community repository

Base URL: `<https://api.logforge.io/v1` > (configurable)

Installation Target: All community templates install to `default/` directory

Consumed Endpoints:

```
GET /api/v1/vendors
→ Returns list of all template vendors

GET /api/v1/vendors/{vendor_id}
→ Returns vendor details and products

```

```
GET /api/v1/vendors/{vendor_id}/{product_id}
→ Returns product details and data sources

GET /api/v1/vendors/{vendor_id}/{product_id}/{data_source_id}
→ Returns data source details and templates

GET /api/v1/vendors/{vendor_id}/{product_id}/{data_source_id}/{template_id}
→ Returns template details and download URL

GET /api/v1/vendors/{vendor_id}/download
→ Downloads complete vendor template package (ZIP)

GET /api/v1/community-templates?page=1&page_size=10&vendor_id=<id>
→ Paginated hierarchical template tree with filtering
```

8.2 Template Discovery & Installation

CLI Commands:

```
# Search templates
logforge templates search "windows"
logforge templates search --vendor microsoft
logforge templates search --product firewall

# List templates with precedence indicators
logforge templates list
# Output:
# NAME                                LOCATION   VERSION  STATUS
# microsoft/windows/eventlog/security     custom     -
(overrides default v1.0.1)
# microsoft/windows/eventlog/application  default    1.0.0
# paloalto/firewall/traffic              default    2.1.0
# acme/custom_app                        custom     -        (custom
only)

logforge templates list --remote           # Remote only
logforge templates list --local            # Installed only
logforge templates list --custom-only      # Only custom templates

# Template information
logforge templates info microsoft/windows/eventlog/security
# Output shows both default and custom versions if both exist

# Install template (always to default/)
logforge templates install microsoft/windows/eventlog/security

# Warning if custom version exists:
# Warning: Custom version exists at
custom/microsoft/windows/eventlog/security
# This will install to default/ but your generators will use the
custom version.
#
# Options:
#   [U] Update custom version from new default
#   [K] Keep custom, install default anyway
#   [C] Cancel
# Choice: _

logforge templates install --vendor microsoft  # Install all from
vendor

# Update templates (only touches default/)
logforge templates update                      # Update all outdated
defaults
logforge templates update microsoft/windows/eventlog/security  #
Specific

# Output:
```

```
# Updating default/microsoft/windows/eventlog/security: 1.0.1 → 1.0.2
# Note: Custom version exists. Run 'logforge templates diff' to review changes.

# Customization commands
logforge templates customize microsoft/windows/eventlog/security
  # Copies default → custom, opens for editing

logforge templates diff microsoft/windows/eventlog/security
  # Shows differences between custom and default versions

logforge templates merge microsoft/windows/eventlog/security
  # Attempts to merge default updates into custom (Git-style)

logforge templates revert microsoft/windows/eventlog/security
  # Removes custom version, falls back to default

# Template creation
logforge templates create custom/acme/myapp
  # Interactive template creator for completely custom templates
```

Version Comparison Output:

```
Template: microsoft/windows/eventlog/security
  Default: 1.0.2 (installed)
  Custom:  - (overriding default)
  Status:  Custom version in use

Template: microsoft/windows/eventlog/application
  Default: 1.0.0
  Remote:  1.0.1
  Status:  Update available

Template: paloalto/firewall/traffic
  Default: 2.1.0
  Remote:  2.1.0
  Status:  Up to date

Template: acme/custom_app
  Custom:  - (custom only)
  Status:  No default version
```

8.3 Template Package Format

Download Format: ZIP file containing template tree structure

Installation: Extracts to `default/` directory

Example Package (`microsoft.zip`):

```
microsoft/
├── windows/
│   ├── eventlog/
│   │   ├── security/
│   │   │   ├── template.j2
│   │   │   ├── metadata.yaml
│   │   │   └── examples/
│   │   │       └── sample_output.xml
│   │   ├── application/
│   │   │   ├── template.j2
│   │   │   └── metadata.yaml
│   │   └── system/
│   │       ├── template.j2
│   │       └── metadata.yaml
│   └── iis/
│       └── access_log/
│           ├── template.j2
│           └── metadata.yaml
└── vendor.yaml
```

vendor.yaml:

```yaml
id: microsoft
name: Microsoft
description: Templates for Microsoft products
version: 1.0.0
updated: 2025-10-15
products:
  - windows
  - iis
  - exchange
  - azure
```

---

9. CLI Interface

9.1 CLI Architecture

Framework: Click or Typer (Python CLI framework)

Connection: All CLI commands are thin wrappers around API calls

API Connection:

```
# Default: localhost
logforge status

# Remote API
logforge --api-url http://other-host:8080 status

# With API key
logforge --api-key abc123 status

# Environment variable
export LOGFORGE_API_URL=http://remote:8080
export LOGFORGE_API_KEY=abc123
logforge status
```

9.2 Complete CLI Command Reference

Template Management Section:

```
#
====================================================================
======
# TEMPLATE MANAGEMENT
#
====================================================================
======

logforge templates list [--local|--remote|--custom-only]
  # List templates with location and version information
  # Shows precedence when both default and custom exist
  # API: GET /api/templates

logforge templates search <query> [--vendor <v>] [--product <p>]
  # Search community templates
  # Community API: GET /api/v1/community-templates?q=<query>

logforge templates info <template_id>
  # Show detailed template information
  # Shows both default and custom versions if both exist
  # API: GET /api/templates/{template_id}

logforge templates install <template_id>
logforge templates install --vendor <vendor_id>
  # Install template(s) from community to default/
  # Warns if custom version already exists
  # Community API: GET /api/v1/vendors/{vendor_id}/download
```

```
logforge templates update [template_id]
  # Update specific or all outdated default/ templates
  # Never touches custom/ templates
  # Community API: GET /api/v1/vendors/...

logforge templates validate <path>
  # Validate local template file
  # Checks: Jinja2 syntax, metadata, format, safety

logforge templates customize <template_id>
  # Copy default template to custom for editing
  # Automatically sets up precedence override
  # Opens editor if --edit flag provided

logforge templates diff <template_id>
  # Show differences between custom and default versions
  # Uses configured diff tool or built-in display

logforge templates merge <template_id>
  # Merge default template changes into custom version
  # Interactive conflict resolution

logforge templates revert <template_id>
  # Remove custom version, revert to using default
  # Prompts for confirmation

logforge templates create <path>
  # Interactive template creator wizard for custom templates
  # Creates in custom/ directory

# Examples:
logforge templates customize microsoft/windows/eventlog/security
logforge templates diff microsoft/windows/eventlog/security
logforge templates merge microsoft/windows/eventlog/security
logforge templates revert microsoft/windows/eventlog/security
```

9.3 CLI Output Formatting

Table Format (default):

```
$ logforge status
NAME              STATE     TEMPLATE                              EVENTS  ERRORS  UPTIME
windows_security  RUNNING   microsoft/windows/eventlog/security  12450   0       59m 12s
palo_traffic      DEGRADED  paloalto/firewall/traffic             8320    3       59m 11s
azure_signin      ERROR     microsoft/azure/signin                0       15      N/A
```

JSON Format (`--output json`):

```
$ logforge status --output json
{
  "generators": [
    {
      "name": "windows_security",
      "state": "RUNNING",
      "template": "microsoft/windows/eventlog/security",
      "events_generated": 12450,
      "errors": 0,
      "uptime": 3552
    }
  ]
}
```

---

10. Deployment & Packaging

10.1 Python Package

Project Structure (see Section 5):

```
logforge/
├── pyproject.toml
├── README.md
├── LICENSE
├── src/logforge/...
└── tests/...
```

pyproject.toml:

```toml
[build-system]
requires = ["setuptools>=68.0", "wheel"]
build-backend = "setuptools.build_meta"

[project]
name = "logforge"
version = "1.0.0"
description = "Synthetic event log generator with template-based architecture"
readme = "README.md"
license = {text = "Apache-2.0"}
authors = [{name = "John Owen", email = "john@ftsc.com"}]
requires-python = ">=3.9"

dependencies = [
    "jinja2>=3.1.0",
    "pyyaml>=6.0",
    "click>=8.1.0",
    "requests>=2.31.0",
    "python-dateutil>=2.8.0",
    "fastapi>=0.109.0",
    "uvicorn>=0.27.0",
    "prometheus-client>=0.19.0",
    "pydantic>=2.5.0",
    "faker>=22.0.0",
]

[project.optional-dependencies]
dev = [
    "pytest>=7.4.0",
    "pytest-cov>=4.1.0",
    "pytest-asyncio>=0.23.0",
    "black>=23.12.0",
    "ruff>=0.1.0",
    "mypy>=1.8.0",
]

[project.scripts]
logforge = "logforge.cli:main"

[project.urls]
Homepage = "https://github.com/yourusername/logforge"
Documentation = "https://docs.logforge.io"
Repository = "https://github.com/yourusername/logforge.git"
```

Installation:

```bash
# From PyPI (when published)
pip install logforge

# From source
git clone https://github.com/yourusername/logforge.git
cd logforge
pip install -e .

```

```
# With dev dependencies
pip install -e ".[dev]"
```

10.2 Docker Deployment

Dockerfile:

```
# Multi-stage build
FROM python:3.11-slim AS builder

WORKDIR /build
COPY pyproject.toml README.md ./
COPY src/ ./src/

RUN pip install --no-cache-dir build && \
    python -m build

FROM python:3.11-slim

# Create logforge user
RUN useradd -m -u 1000 logforge

# Install package
COPY --from=builder /build/dist/*.whl /tmp/
RUN pip install --no-cache-dir /tmp/*.whl && \
    rm /tmp/*.whl

# Create directories
RUN mkdir -p /home/logforge/.logforge/templates && \
    chown -R logforge:logforge /home/logforge

USER logforge
WORKDIR /home/logforge

# Expose API port
EXPOSE 8080

# Volume for config and output
VOLUME ["/home/logforge/.logforge", "/var/log/logforge"]

# Health check
HEALTHCHECK --interval=30s --timeout=3s --start-period=10s \
  CMD curl -f http://localhost:8080/api/health || exit 1

# Start with API enabled
CMD ["logforge", "api", "start", "--host", "0.0.0.0"]
```

docker-compose.yml:

```
version: '3.8'

services:
  logforge:
    build: .
    image: logforge:latest
    container_name: logforge
    ports:
      - "8080:8080"
    volumes:
      - ./config:/home/logforge/.logforge
      - ./logs:/var/log/logforge
    environment:
      - LOGFORGE_LOG_LEVEL=INFO
      - LOGFORGE_API_HOST=0.0.0.0
      - LOGFORGE_API_PORT=8080
    restart: unless-stopped
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:8080/api/health"]
      interval: 30s
      timeout: 3s
```

```
      retries: 3
```

Quick Start:

```
# Build and run
docker-compose up -d

# Initialize configuration
docker exec logforge logforge init

# Start generators
docker exec logforge logforge start

# Check status
docker exec logforge logforge status

# View logs
docker-compose logs -f
```

10.3 Systemd Integration (Optional)

Service Unit (`/etc/systemd/system/logforge.service`):

```
[Unit]
Description=LogForge Synthetic Event Generator
After=network.target

[Service]
Type=simple
User=logforge
Group=logforge
WorkingDirectory=/home/logforge
ExecStart=/usr/local/bin/logforge api start
Restart=on-failure
RestartSec=10s

# Environment
Environment="LOGFORGE_CONFIG=/etc/logforge/config.yaml"

# Logging
StandardOutput=journal
StandardError=journal
SyslogIdentifier=logforge

[Install]
WantedBy=multi-user.target
```

Installation:

```
# Install systemd service
sudo cp logforge.service /etc/systemd/system/
sudo systemctl daemon-reload
sudo systemctl enable logforge
sudo systemctl start logforge

# Check status
sudo systemctl status logforge

# View logs
sudo journalctl -u logforge -f
```

---

11. Development Phases

Phase 1: Foundation + API Server Core (Week 1-2)

Deliverables:

- ✅ Project structure and packaging setup
- ✅ Configuration management (YAML loader/validator)
- ✅ FastAPI server skeleton with basic endpoints
- ✅ Health check endpoint (`GET /api/health`)
- ✅ Logging infrastructure setup
- ✅ CLI framework with `init` , `config` commands

Acceptance Criteria:

- `logforge init` creates default config and directory structure
- `logforge config show` displays current configuration
- API server starts and responds to health checks
- Application logging works correctly

Phase 2: Entity Registry + API (Week 3)

Deliverables:

- ✅ Entity registry storage (YAML file-based)
- ✅ In-memory cache with auto-save
- ✅ Registry functions for template access
- ✅ Entity API endpoints (`GET/POST /api/entities`)
- ✅ CLI entity management commands
- ✅ Entity validation logic

Acceptance Criteria:

- Entities can be added/imported/exported via CLI
- Registry functions work correctly (`get_random_user()` , etc.)
- Entity validation catches errors
- API endpoints return proper entity data

Phase 3: Template System + Community Integration (Week 4-5)

Deliverables:

- ✅ Jinja2 template loader and renderer
- ✅ Faker library integration
- ✅ Template metadata parser and validator
- ✅ Local template discovery (filesystem scanning)
- ✅ Community API client (HTTP client for remote API)
- ✅ Template API endpoints (`GET /api/templates`)
- ✅ CLI template commands (list, search, install, validate)

Acceptance Criteria:

- Templates render correctly with registry functions and faker
- Templates can be discovered locally and remotely
- Templates can be installed from community repository

- Template validation catches syntax and metadata errors

Phase 4: Event Generation Engine + API (Week 6-7)

Deliverables:

- ✅ Generator class with state machine
- ✅ ThreadPoolExecutor setup with dynamic sizing
- ✅ Frequency control (time-based variation)
- ✅ Generator lifecycle management
- ✅ Smart error recovery logic
- ✅ Generator API endpoints (`GET/POST /api/generators`)
- ✅ CLI generator commands (start, stop, status)

Acceptance Criteria:

- Generators start/stop correctly via CLI and API
- Multiple generators run concurrently
- Frequency variation works (time of day, day of week)
- State transitions work correctly
- Error recovery behaves as specified

Phase 5: Output Handlers + Metrics (Week 8-9)

Deliverables:

- ✅ Base output handler class
- ✅ File output with rotation
- ✅ Console output (JSON and text)
- ✅ HTTP output with batching
- ✅ TCP and Syslog outputs
- ✅ Retry logic with exponential backoff
- ✅ Event buffering during outages
- ✅ Prometheus metrics endpoint
- ✅ Status API endpoint with detailed info

Acceptance Criteria:

- Events written to all output types correctly
- File rotation works (size and time-based)
- Retry logic recovers from transient failures
- Metrics endpoint returns valid Prometheus format
- Buffering prevents event loss during outages

Phase 6: Deployment + Documentation (Week 10)

Deliverables:

- ✅ Dockerfile with multi-stage build
- ✅ docker-compose.yml with examples
- ✅ Systemd service unit (optional)

- ✅ Complete README with quick start
- ✅ API documentation (OpenAPI/Swagger)
- ✅ Template development guide
- ✅ Example templates (2-3 bundled)
- ✅ PyPI package publishing

Acceptance Criteria:

- Docker image builds and runs correctly
- docker-compose provides working example
- Documentation is complete and accurate
- Package installs cleanly via pip
- Example templates work out of the box

---

12. Testing Strategy

12.1 Test Categories

Unit Tests:

- Template rendering logic
- Entity registry functions
- Configuration parsing
- Output handler formatting
- API endpoint responses

Integration Tests:

- Generator lifecycle with real templates
- Output handler retry logic
- Community API client
- CLI command execution
- Multi-generator concurrency

End-to-End Tests:

- Complete workflow: init → install templates → start generators → verify output
- Docker deployment testing
- API authentication testing

12.2 Test Coverage Requirements

- Minimum 80% code coverage
- 100% coverage for critical paths (generator lifecycle, error handling)
- All API endpoints tested
- All CLI commands tested

12.3 Testing Tools

```
[project.optional-dependencies]
dev = [
    "pytest>=7.4.0",           # Test framework
```

```
    "pytest-cov>=4.1.0",       # Coverage reporting
    "pytest-asyncio>=0.23.0",  # Async test support
    "pytest-mock>=3.12.0",     # Mocking
    "httpx>=0.26.0",           # HTTP client for API testing
]
```

---

13. Key Design Decisions Summary

Decision	Choice	Rationale
API Architecture	API-first with embedded FastAPI	Enables remote control, future web UI, consistent interface
API Server Lifecycle	Single process, background thread	Simplicity for OSS version, optional disable for edge cases
API Authentication	Optional (disabled by default)	Localhost trust for ease of use, optional key for security
Threading Model	ThreadPoolExecutor, dynamic sizing	Python standard library, auto-scales to CPU, proven approach
Entity Storage	File-based YAML	Simple, human-editable, version-controllable, no database dependency
Template Engine	Jinja2 + Faker	Industry standard, powerful, extensive fake data generation
Configuration Format	YAML	Human-readable, widely adopted, good for structured config
Error Recovery	Smart retry with state machine	Balances robustness with clear failure modes
Output Retry	Exponential backoff, unlimited by default	Resilient to transient failures, configurable limits
CLI Framework	Click or Typer	Python standard, excellent DX, auto-generated help

Metrics Format	Prometheus-compatible	Industry standard for monitoring, tool compatibility
License	Apache 2.0	Attribution required, patent protection, enterprise-friendly
Template Examples	Include 2-3 basic templates	Immediate functionality testing, clear examples
Config Initialization	Default + optional wizard	Quick start with sensible defaults, wizard for customization

---

14. Non-Functional Requirements

14.1 Performance Targets

- **Event Generation**: 1000+ events/second per generator on modest hardware (4-core, 8GB RAM)
- **API Response Time**: <100ms for status endpoints, <500ms for generator operations
- **Memory Usage**: <500MB for 10 concurrent generators
- **Startup Time**: <5 seconds to API ready
- **Template Rendering**: <10ms per event

14.2 Reliability

- **Uptime**: Generators continue running during configuration reloads
- **Data Loss**: Zero data loss during normal operation, buffering during outages
- **Recovery**: Automatic recovery from transient failures within 5 minutes

14.3 Maintainability

- **Code Quality**: Type hints for all public APIs, docstrings for all modules
- **Logging**: Comprehensive logging at appropriate levels
- **Error Messages**: Clear, actionable error messages with context
- **Documentation**: Complete API docs, template guides, deployment instructions

14.4 Security

- **File Permissions**: Config and entity files readable only by owner
- **API Security**: Optional API key authentication
- **Template Safety**: No file system access or code execution in templates
- **Input Validation**: All user inputs validated before processing

---

15. Appendix

15.1 Python Module Structure

```
src/logforge/
├── __init__.py              # Package initialization, version
├── __main__.py              # Entry point for python -m logforge
│
├── cli/                     # CLI Interface
│   ├── __init__.py
│   ├── main.py              # Main CLI entry point
│   ├── generators.py        # Generator commands
│   ├── templates.py         # Template commands
│   ├── entities.py          # Entity commands
│   └── config.py            # Config commands
│
├── core/                    # Core Engine
│   ├── __init__.py
│   ├── engine.py            # Main generation engine
│   ├── generator.py         # Generator class with state machine
│   ├── config.py            # Configuration management
│   └── frequency.py         # Frequency control logic
│
├── templates/               # Template System
│   ├── __init__.py
│   ├── loader.py            # Template discovery and loading
│   ├── renderer.py          # Jinja2 rendering with context
│   ├── validator.py         # Template validation
│   └── filters.py           # Custom Jinja2 filters
│
├── entities/                # Entity Registry
│   ├── __init__.py
│   ├── registry.py          # Registry management, CRUD
│   ├── functions.py         # Template-accessible functions
│   ├── storage.py           # File persistence layer
│   └── validator.py         # Entity validation
│
├── outputs/                 # Output Handlers
│   ├── __init__.py
│   ├── base.py              # Base handler abstract class
│   ├── file.py              # File output with rotation
│   ├── console.py           # Console output
│   ├── http.py              # HTTP output with batching
│   ├── tcp.py               # TCP socket output
│   └── syslog.py            # Syslog protocol output
│
├── api/                     # Management API
│   ├── __init__.py
│   ├── server.py            # FastAPI application
│   ├── endpoints/           # API endpoint modules
│   │   ├── __init__.py
│   │   ├── health.py        # Health and status endpoints
│   │   ├── generators.py    # Generator management endpoints
│   │   ├── templates.py     # Template endpoints
│   │   └── entities.py      # Entity endpoints
│   ├── models.py            # Pydantic models for requests/responses
│   └── auth.py              # Optional API key authentication
│
├── community/               # Community Integration
│   ├── __init__.py
│   └── client.py            # HTTP client for community API
│
└── utils/                   # Utilities
    ├── __init__.py
    ├── logging.py           # Logging setup and configuration
    ├── metrics.py           # Prometheus metrics collection
    └── validation.py        # Common validation functions
```

15.2 Key Classes and Interfaces

Generator Class:

```python
class Generator:
    """Manages event generation lifecycle for a single generator."""
```

```
    def __init__(self, name: str, config: GeneratorConfig):
        self.name = name
        self.state = GeneratorState.STOPPED
        self.template = None
        self.outputs = []

    async def start(self) -> None:
        """Start event generation."""

    async def stop(self) -> None:
        """Stop event generation gracefully."""

    async def _generate_loop(self) -> None:
        """Main generation loop."""

    def _calculate_rate(self) -> float:
        """Calculate current event rate based on time/day."""
```

Output Handler Interface:

```python
class OutputHandler(ABC):
    """Abstract base class for output handlers."""

    @abstractmethod
    async def write(self, event: str) -> None:
        """Write a single event."""

    @abstractmethod
    async def write_batch(self, events: List[str]) -> None:
        """Write a batch of events."""

    @abstractmethod
    async def close(self) -> None:
        """Clean up resources."""
```

15.3 Environment Variables

```bash
# API Configuration
LOGFORGE_API_URL=http://127.0.0.1:8080
LOGFORGE_API_KEY=abc123

# Configuration File
LOGFORGE_CONFIG=/path/to/config.yaml

# Logging
LOGFORGE_LOG_LEVEL=INFO
LOGFORGE_LOG_FILE=/var/log/logforge/logforge.log

# Entity Registry
LOGFORGE_ENTITIES_PATH=/path/to/entities.yaml

# Templates
LOGFORGE_TEMPLATES_PATH=/path/to/templates
LOGFORGE_COMMUNITY_API_URL=https://api.logforge.io/v1
```

---

16. Success Criteria

16.1 MVP Release Checklist

- ✅ User can `pip install logforge` and get working system
- ✅ `logforge init` creates sensible defaults
- ✅ User can install templates from community
- ✅ Multiple generators run concurrently

- ✅ Events written to file with rotation
- ✅ API provides health checks and status
- ✅ CLI provides all core functionality
- ✅ Docker deployment works out of the box
- ✅ Documentation complete and accurate
- ✅ 80%+ test coverage

16.2 User Experience Goals

- **First Run Success**: User goes from install to generating logs in <5 minutes
- **Template Discovery**: User finds and installs relevant templates easily
- **Error Clarity**: When something fails, user knows exactly what and how to fix
- **Monitoring**: User can check system health and generator status at a glance
- **Customization**: User can create custom templates without coding